

Deploy Multi-Container Application Using Docker Compose

Introduction to Containerization

Containerization is a lightweight virtualization technology that packages applications and their dependencies into isolated environments called containers. These containers ensure that applications run consistently across different environments such as development, testing, and production.

Containerization reduces dependency conflicts, improves portability, and enables efficient resource utilization.

1. Introduction to Docker

Docker is an open-source containerization platform that enables developers to build, package, deploy, and run applications within lightweight and isolated containers. By bundling an application along with its dependencies, libraries, and configuration files, Docker ensures consistent performance across different environments, such as development, testing, and production.

Docker offers several advantages, including application portability, faster deployment, improved scalability, environment consistency, and efficient utilization of system resources. Due to these benefits, Docker has become a key technology in modern DevOps practices, cloud computing, and application deployment.

2. Importance, Benefits, and Applications of Docker

Docker was developed to address common challenges in traditional software deployment, such as dependency conflicts, environment inconsistencies, complex application migration, and high resource consumption. By packaging applications along with their required dependencies into lightweight containers, Docker ensures that applications run consistently across different environments.

Docker offers several advantages, including lightweight architecture, portability, rapid deployment, scalability, efficient resource utilization, environment consistency, and simplified application maintenance. Since containers share the host operating system kernel, they require fewer resources and start significantly faster than traditional virtual machines.

Due to these benefits, Docker has become a widely adopted technology in modern software development and DevOps practices. It is extensively used for deploying web applications, building microservices-based architectures, implementing CI/CD and DevOps pipelines, creating testing and development environments, running cloud-native applications, and supporting container orchestration platforms such as Kubernetes.

Overall, Docker simplifies application deployment, improves operational efficiency, and enables organizations to deliver software faster and more reliably.

What is a Container?

A **container** is a lightweight and portable software package that includes an application along with its runtime environment, system libraries, dependencies, and configuration files required for execution.

Containers provide a consistent environment across different systems, ensuring that applications run reliably regardless of where they are deployed. A container is essentially a runnable instance of a Docker image.

5. Architecture of Docker

Docker follows a Client–Server Architecture where the Docker Client communicates with the Docker Daemon to build, run, and manage containers. Docker enables applications to be packaged with all their dependencies into lightweight, portable containers.

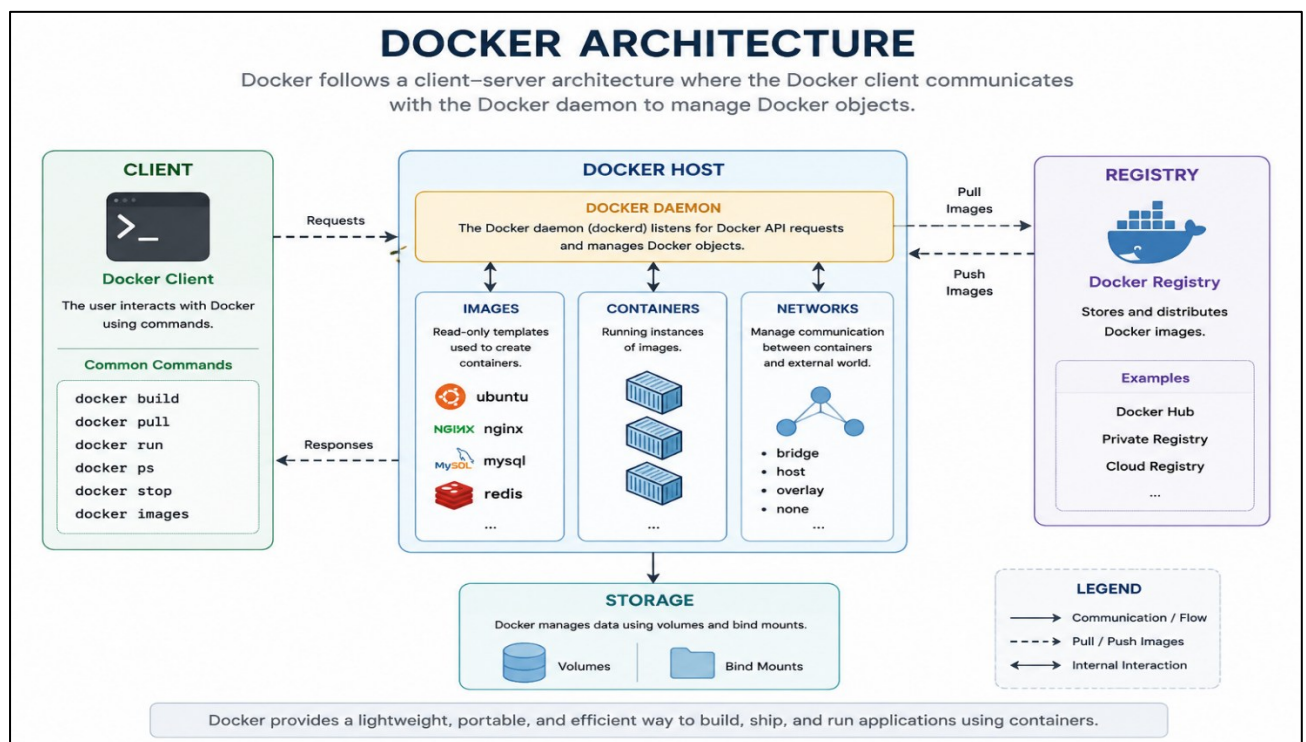


Figure: Architecture Diagram of Docker.

Components of Docker Architecture

Component	Description	Function
Docker Client	User interface to interact with Docker.	Sends commands such as <code>docker build</code> , <code>docker run</code> , and <code>docker pull</code> to Docker Daemon.
Docker Daemon (dockerd)	Background service running on the host machine.	Builds, runs, manages containers, images, networks, and volumes.
Docker Host	Physical or virtual machine where Docker runs.	Provides resources such as CPU, RAM, Storage, and Network.

Docker Image	Read-only template containing application code and dependencies.	Used to create containers.
Docker Container	Running instance of a Docker image.	Executes the application in an isolated environment.
Docker Registry	Storage repository for Docker images.	Stores and distributes images. Example: Docker Hub.
Docker Hub	Public cloud registry maintained by Docker.	Allows users to push and pull images.
Docker Network	Communication layer for containers.	Enables container-to-container and external communication.
Docker Volume	Persistent storage mechanism.	Preserves data even after containers are deleted.
Docker Engine	Core Docker runtime environment.	Combines Docker Client, Docker Daemon, and APIs.

Table: Components of Docker.

4. Working of Docker Architecture

Step	Process	Explanation
1	User Command	User executes a Docker command from the Docker Client.
2	Request Sent	Docker Client sends API request to Docker Daemon.
3	Image Check	Daemon checks whether the required image exists locally.
4	Image Download	If image is unavailable, it is pulled from Docker Registry.
5	Container Creation	Docker Daemon creates a container using the image.
6	Resource Allocation	CPU, memory, storage, and networking are assigned.
7	Application Execution	Container starts and runs the application.
8	Monitoring	Docker Daemon continuously manages container lifecycle.

Table: Working of Docker

5. Important Docker Terms

Term	Meaning
Dockerfile	Text file containing instructions to build Docker images.
Build	Process of creating an image from a Dockerfile.
Pull	Downloading an image from a registry.
Push	Uploading an image to a registry.

Tag	Version identifier for Docker images.
Repository	Collection of related Docker images.
Layer	Individual image component reused across images.
Namespace	Linux feature providing process isolation.
cgroups	Linux feature controlling CPU, memory, and resource allocation.
Bind Mount	Direct mapping of host directory into container.
Volume	Docker-managed persistent storage.

Table: Docker Terms

7. Difference Between Docker Containers and Virtual Machines

Feature	Docker Container	Virtual Machine
OS Kernel	Shared Host Kernel	Separate Guest OS
Startup Time	Seconds	Minutes
Resource Usage	Low	High
Size	MBs	GBs
Performance	Near Native	Additional Hypervisor Overhead
Isolation	Process-Level	Hardware-Level

Table: Docker Containers Vs Virtual Machines

Docker Workflow

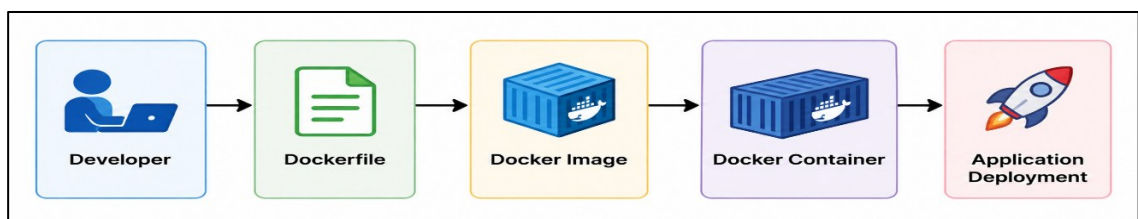


Figure: Docker Workflow

Project Overview

This project demonstrates the deployment of a multi-container application using Docker and Docker Compose on VMware Cloud Director.

The application consists of:

- Frontend Container (Nginx)
- Backend Container (Node.js)
- MySQL Container
- Redis Container

Docker Compose was used to deploy and manage multiple containers as a single application stack.

Project Objectives

- To deploy Ubuntu virtual machines.
- To install and configure Docker and Docker Compose.
- To build and deploy containerized applications.
- To implement Docker networking and persistent storage.
- To verify container communication and accessibility.
- To enable application deployment through a public IP address.

11. System Specifications

Component	Details
Platform	VMware Cloud Director
Operating System	Ubuntu 22.04 LTS
Docker Version	Latest
Orchestration	Docker Compose
Web Server	Nginx
Backend	Node.js
Database	MySQL 8
Cache	Redis
SSH Port	2232

Table: System Specifications

Project Architecture

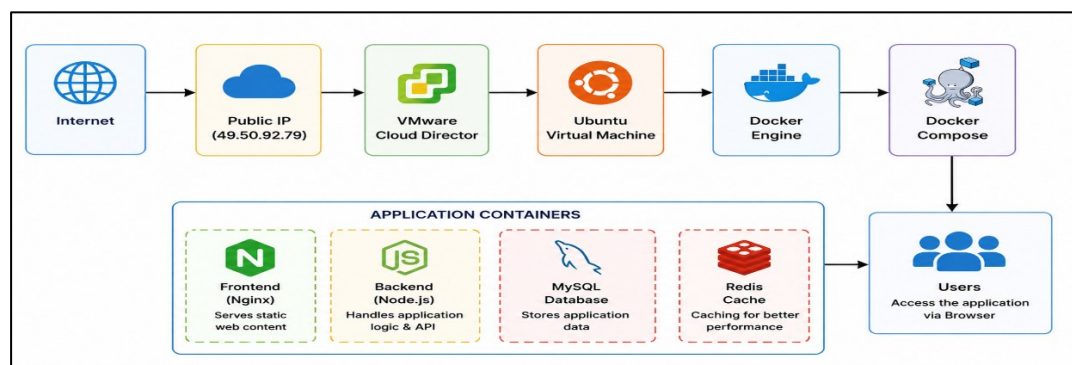


Figure: Project Architecture

IMPLEMENTATION

1.Create Virtual Machines

VM Name	CPU	RAM	Disk
Docker-App	4	10 GB	60 GB
Mysql-Server	4	10 GB	50 GB
Backup-Server	4	10 GB	50 GB

Table: VMs Configuration

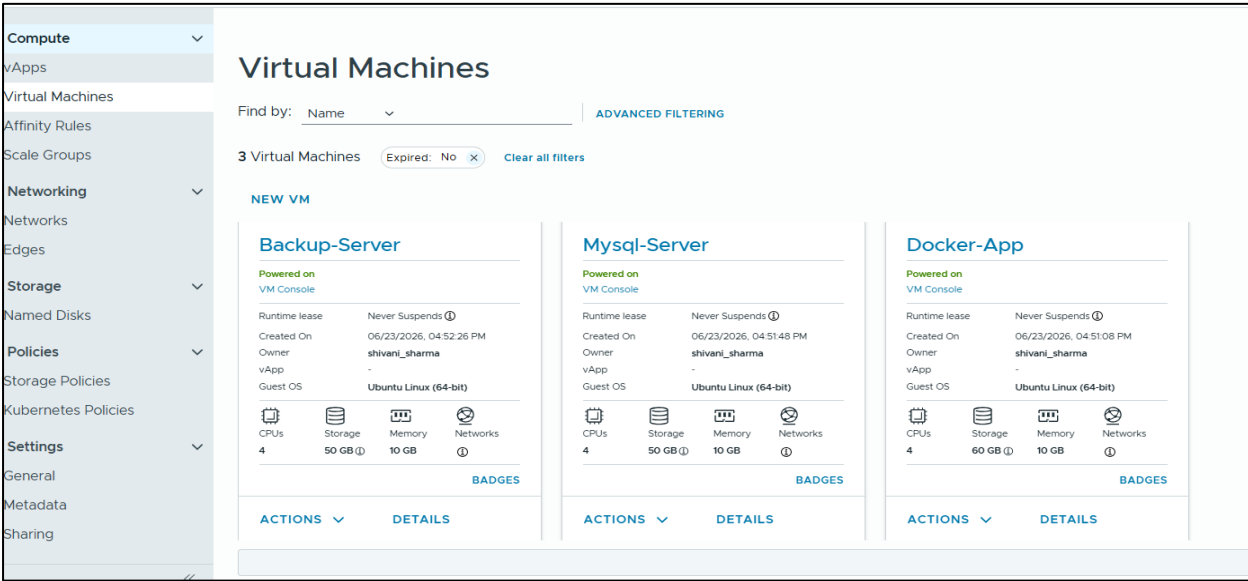


Figure: VMs Configuration

2.Update Ubuntu Server

Step	Command
Update Package List	apt update
Upgrade Packages	apt upgrade -y

Table: Server Update Commands

3. Install Docker

Step	Command
Install Docker	curl -fsSL https://get.docker.com sh
Verify Installation	docker --version
Check Service	systemctl status docker
Enable Service	systemctl enable docker

Table: Docker Installation Commands

```

root@Docker-App:~# docker --version
Docker version 29.6.0, build fb59821
root@Docker-App:~# systemctl enable docker
systemctl start docker
systemctl status docker
Synchronizing state of docker.service with SysV service script with /lib/systemd
/systemd-sysv-install.
Executing: /lib/systemd/systemd-sysv-install enable docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/lib/systemd/system/docker.service; enabled; vendor preset:
   Active: active (running) since Tue 2026-06-23 11:48:40 UTC; 35s ago
   TriggeredBy: ● docker.socket
     Docs: https://docs.docker.com
    Main PID: 54811 (dockerd)
      Tasks: 10
     Memory: 24.6M
        CPU: 610ms
    CGroup: /system.slice/docker.service
            └─54811 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/con>

Jun 23 11:48:39 docker-app dockerd[54811]: time="2026-06-23T11:48:39.672319880Z>
Jun 23 11:48:39 docker-app dockerd[54811]: time="2026-06-23T11:48:39.712057113Z>
Jun 23 11:48:39 docker-app dockerd[54811]: time="2026-06-23T11:48:39.727412971Z>
Jun 23 11:48:40 docker-app dockerd[54811]: time="2026-06-23T11:48:40.101500738Z>
Jun 23 11:48:40 docker-app dockerd[54811]: time="2026-06-23T11:48:40.110400228Z>
Jun 23 11:48:40 docker-app dockerd[54811]: time="2026-06-23T11:48:40.110584513Z>
Jun 23 11:48:40 docker-app dockerd[54811]: time="2026-06-23T11:48:40.259886762Z>
Jun 23 11:48:40 docker-app dockerd[54811]: time="2026-06-23T11:48:40.271212502Z>

```

Figure: Docker Installed.

4. Install Docker Compose

Step	Command
Install Compose Plugin	apt install docker-compose-plugin -y
Verify Installation	docker compose version

Table: Docker Compose Installation

```

root@Docker-App:~# apt install docker-compose-plugin -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
docker-compose-plugin is already the newest version (5.1.4-1~ubuntu.22.04~jammy).
The following packages were automatically installed and are no longer required:
  libfwupd2 libfwupdplugin5 libgcab-1.0-0 libgusb2 libmbim-glib4 libmbim-proxy libm
Use 'apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
root@Docker-App:~# docker compose version
Docker Compose version v5.1.4
root@Docker-App:~# █

```

Figure: Docker Compose Installed.

5. Create Project Directory

Step	Command
Create Project Folder	mkdir /root/docker-project
Navigate Directory	cd /root/docker-project
Create Application Folders	mkdir frontend backend

Table: Project Directory Creation

```
root@Docker-App:~# mkdir /root/docker-project

cd /root/docker-project
root@Docker-App:~/docker-project# mkdir frontend backend
```

Figure: Project Directory Creation

6. Create Frontend

Step	Command
Navigate Directory	cd frontend
Create HTML File	nano index.html
Create Dockerfile	nano Dockerfile

Frontend Dockerfile:

```
FROM nginx
COPY . /usr/share/nginx/html
```

Table: Frontend Container Configuration

```
root@Docker-App:~/docker-project# cd /root/docker-project/frontend
root@Docker-App:~/docker-project/frontend# nano index.html
root@Docker-App:~/docker-project/frontend# nano Dockerfile
root@Docker-App:~/docker-project/frontend# cat /root/docker-project/frontend/Dockerfile
FROM nginx
COPY . /usr/share/nginx/html
root@Docker-App:~/docker-project/frontend# █
```

Figure: Frontend Container Configuration

6. Create Backend

Step	Command
Navigate Directory	cd ../backend
Create Application File	nano server.js
Create Package File	nano package.json
Create Dockerfile	nano Dockerfile

Backend Dockerfile:

```
FROM node:20
WORKDIR /app
COPY . .
RUN npm install
EXPOSE 3000
```


CMD ["node","server.js"]

Table: Backend Container Configuration

```
root@Docker-App:~/docker-project/frontend# cd /root/docker-project/backend
root@Docker-App:~/docker-project/backend# nano server.js
root@Docker-App:~/docker-project/backend# nano package.json
root@Docker-App:~/docker-project/backend# nano Dockerfile
root@Docker-App:~/docker-project/backend# cd /root/docker-project/backend

ls
Dockerfile  package.json  server.js
```

Figure: Backend Container Configuration

7. Create Docker Compose File

Step	Command
Navigate Project Directory	cd /root/docker-project
Create Compose File	nano docker-compose.yml

Services configured:

- Frontend
- Backend
- MySQL
- Redis

Table: Docker Compose Configuration

8. Build Images

Command	Purpose
docker compose build	Build all container images

Table: Image Build Command

```
=> [backend 3/4] COPY . .
=> [backend 4/4] RUN npm install
=> [backend] exporting to image
=> => exporting layers
=> => exporting manifest sha256:75300ad6cdb8878fe61f58c7d177
=> => exporting config sha256:83fe269f51d34491ff51031ac7ea0d
=> => exporting attestation manifest sha256:85b19b3ada80ccc3
=> => exporting manifest list sha256:beb5de66b4a6a3fb487b5ee
=> => naming to docker.io/library/docker-project-backend:lat
=> => unpacking to docker.io/library/docker-project-backend:
=> [backend] resolving provenance for metadata file
[+] build 2/2
✔ Image docker-project-frontend Built
✔ Image docker-project-backend Built
root@Docker-App:~/docker-project#
```

Figure: Image Build

9. Deploy Containers

Command	Purpose
docker compose up -d	Start all containers in detached mode

Table: Container Deployment Command

```
root@Docker-App:~/docker-project# docker compose up -d
[+] up 28/28
  ✓ Image mysql:8 Pulled
  ✓ Image redis Pulled
  ✓ Network docker-project_default Created
  ✓ Volume docker-project_mysql-data Created
  ✓ Container frontend Started
  ✓ Container backend Started
  ✓ Container redis Started
  ✓ Container mysql Started
root@Docker-App:~/docker-project#
```

Figure: Container Deployment

10. Verify Containers

Command	Purpose
docker ps	List running containers
docker images	List Docker images
docker compose ps	View compose services

Table: Container Verification Commands

```
root@docker-app: ~
CONTAINER ID   NAME          CPU %     MEM USAGE / LIMIT   MEM %     NET I/O       BLOCK I/O     PIDS
76f249fc411e   frontend     0.00%    5.496MiB / 9.689GiB  0.06%    38.1kB / 91.8kB  0B / 24.6kB   5
b20b636259b3   redis        0.25%    6.109MiB / 9.689GiB  0.06%    3.61kB / 126B   0B / 0B        9
b769019d385f   mysql        0.44%    425MiB / 9.689GiB   4.28%    3.21kB / 126B   131MB / 566MB  34
827702dc0383   backend      0.00%    12.95MiB / 9.689GiB  0.13%    3.33kB / 126B   0B / 0B        7
```

```
root@Docker-App:~/docker-project# docker ps
CONTAINER ID   IMAGE                                COMMAND
b20b636259b3   redis                               "docker-entrypoint.s..."
b769019d385f   mysql:8                             "docker-entrypoint.s..."
827702dc0383   docker-project-backend              "docker-entrypoint.s..."
d2f6e31c45dd   docker-project-frontend             "/docker-entrypoint...."
root@Docker-App:~/docker-project#
```

```
root@docker-app:~# docker images
IMAGE                                ID                                DISK USAGE
docker-project-backend:latest        beb5de66b4a6                     1.59GB
docker-project-frontend:latest       98f3f401bab4                     238MB
hello-world:latest                   96498fffd522e                    25.9kB
mysql:8                               563602a18ffd                     1.12GB
redis:latest                          0a972391db0b                     208MB
root@docker-app:~#
```

Figure: Container Verification

11. Docker Management Commands

Command	Purpose
docker logs frontend	View frontend logs
docker logs backend	View backend logs
docker network ls	List Docker networks
docker volume ls	List Docker volumes
docker stats	Monitor container resources
docker inspect backend	View container details

Table: Docker Monitoring Commands

```
root@docker-app:~# docker logs frontend
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/06/23 12:24:18 [notice] 1#1: using the "epoll" event method
2026/06/23 12:24:18 [notice] 1#1: nginx/1.31.2
2026/06/23 12:24:18 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/06/23 12:24:18 [notice] 1#1: OS: Linux 5.15.0-43-generic
2026/06/23 12:24:18 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1024:524288
2026/06/23 12:24:18 [notice] 1#1: start worker processes
2026/06/23 12:24:18 [notice] 1#1: start worker process 29
2026/06/23 12:24:18 [notice] 1#1: start worker process 30
2026/06/23 12:24:18 [notice] 1#1: start worker process 31
2026/06/23 12:24:18 [notice] 1#1: start worker process 32
49.50.73.97 - - [23/Jun/2026:12:24:31 +0000] "GET / HTTP/1.1" 200 1779 "-" "Mozilla/5.0 (Windows
Chrome/147.0.0.0 Safari/537.36" "-"
45.198.224.186 - - [23/Jun/2026:12:43:38 +0000] "GET / HTTP/1.1" 200 1779 "-" "Mozilla/5.0" "-"
176.65.139.66 - - [23/Jun/2026:12:43:46 +0000] "GET / HTTP/1.0" 200 1779 "-" "Shodan-Pull/1.0"
151.243.11.35 - - [23/Jun/2026:12:58:51 +0000] "GET / HTTP/1.1" 400 157 "-" "curl/7.68.0" "-"
176.65.139.87 - - [23/Jun/2026:13:16:42 +0000] "GET / HTTP/1.1" 200 1779 "-" "-" "-"
```

```
^Croot@docker-app:~# docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
476bfc854024        bridge              bridge              local
8dce10a24ca6        docker-project_default bridge              local
a167bbdb8d42        host                host                local
caea5a008541        none                null                local
root@docker-app:~# docker volume ls
DRIVER              VOLUME NAME
local               docker-project_mysql-data
root@docker-app:~#
```

Figure: Docker Monitoring

11. Access Application

Service	URL
Frontend	http://49.50.92.79

Backend	http://49.50.92.79:3000
API Endpoint	http://49.50.92.79:3000/api

Table: Application Access URLs

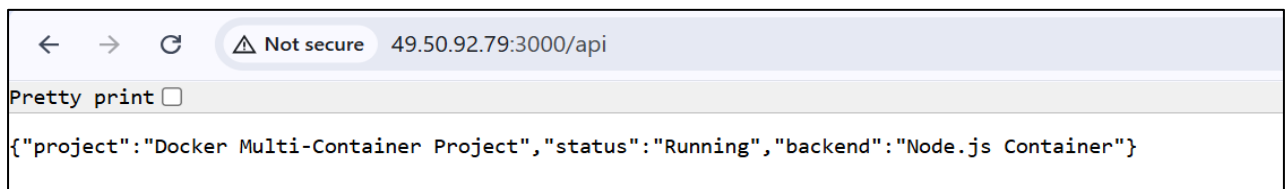
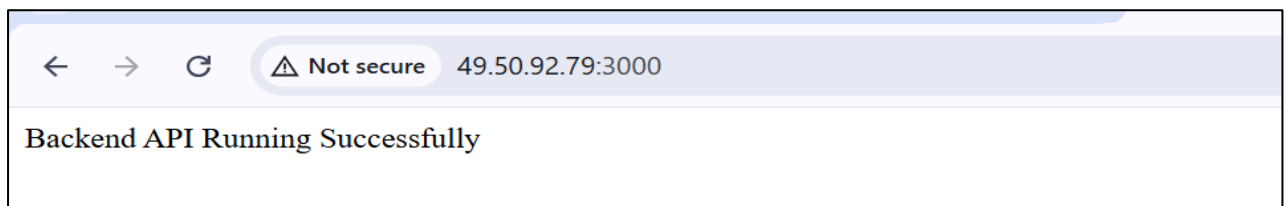


Figure: Application Access

Challenges Encountered

Issue	Resolution
Docker installation failed	Reinstalled Docker
UTF-8 encoding issue	Added UTF-8 tag
Git authentication failed	Created PAT
Git push rejected	Pulled remote changes
Container rebuild issue	Used --build option

Table: Challenges Encountered

Project Repository

GitHub Repository:

<https://github.com/2121-shivi/docker-multi-container-project>

Conclusion

The Docker Multi-Container Project was successfully implemented using VMware Cloud Director and Ubuntu Linux. Docker Compose was utilized to deploy and manage multiple containers, including Nginx, Node.js, MySQL, and Redis.

The project provided hands-on experience with Docker images, containers, networking, volumes, and multi-container application deployment. It demonstrated the practical implementation of containerization technology and modern application deployment practices, highlighting the efficiency, scalability, and portability offered by Docker-based environments.